1.Implement a Stack using Queue data structure.

code:

```
#include <stdio.h>
#define MAX 100
int queue[MAX];
int front = 0, rear = -1;
void push(int x)
{
    int i;
    if (rear == MAX - 1)
    {
        printf("Stack Overflow\n");
        return;
    }
    for (i = rear; i >= front; i--)
        queue[i + 1] = queue[i];
    queue[front] = x;
    rear++;
    printf("Pushed: %d\n", x);
}
void pop()
{
    if (rear < front)
    {
        printf("Stack Underflow\n");
        return;
    }
    printf("Popped: %d\n", queue[front]);
    front++;
}
void display()
{
    int i;
    if (rear < front)
    {
        printf("Stack is empty\n");
        return;
    }
    printf("Stack elements: ");
    for (i = front; i <= rear; i++)
        printf("%d ", queue[i]);
    printf("\n");
}
int main()
{
    int choice, value;
    while (1)
    {
        printf("\n1. Push\n2. Pop\n3. Display\n4. Exit\n");
```

```
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
            case 1:
                printf("Enter value: ");
                scanf("%d", &value);
                push(value);
                break;
            case 2:
                pop();
                break;
            case 3:
                display();
                break;
            case 4:
                return 0;
            default:
                printf("Invalid choice\n");
        }
    }
}
```

output:

```
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 1
Enter value: 10
Pushed: 10

1. Push
2. Pop
3. Display
4. Exit
Enter choice: 1
Enter value: 20
Pushed: 20

1. Push
2. Pop
3. Display
4. Exit
Enter choice: 1
Enter value: 30
Pushed: 30

1. Push
2. Pop
3. Display
4. Exit
```

```
Enter choice: 3
Stack elements: 30 20 10

1. Push
2. Pop
3. Display
4. Exit
Enter choice: 2
Popped: 30
```

2. Implement a Queue using Stack data structure.

code:

```
#include <stdio.h>
#define MAX 100
int stack1[MAX], stack2[MAX];
int top1 = -1, top2 = -1;
void enqueue(int x)
{
    if (top1 == MAX - 1)
    {
        printf("Queue Overflow\n");
        return;
    }
    stack1[++top1] = x;
    printf("Inserted: %d\n", x);
}
void dequeue()
{
    int i;
    if (top1 == -1 && top2 == -1)
    {
        printf("Queue Underflow\n");
        return;
    }
    if (top2 == -1)
    {
        while (top1 != -1)
            stack2[++top2] = stack1[top1--];
    }
    printf("Deleted: %d\n", stack2[top2--]);
}
void display()
{
    int i;
    if (top1 == -1 && top2 == -1)
    {
        printf("Queue is empty\n");
        return;
    }
    printf("Queue elements: ");
    for (i = top2; i >= 0; i--)
```

```
        printf("%d ", stack2[i]);
    for (i = 0; i <= top1; i++)
        printf("%d ", stack1[i]);
    printf("\n");
}
int main()
{
    int choice, value;
    while (1)
    {
        printf("\n1. Enqueue\n2. Dequeue\n3. Display\n4. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
            case 1:
                printf("Enter value: ");
                scanf("%d", &value);
                enqueue(value);
                break;
            case 2:
                dequeue();
                break;
            case 3:
                display();
                break;
            case 4:
                return 0;
            default:
                printf("Invalid choice\n");
        }
    }
}
```

output:

```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 1
Enter value: 10
Inserted: 10

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 1
Enter value: 20
Inserted: 20

1. Enqueue
2. Dequeue
```

```
3. Display
4. Exit
Enter choice: 1
Enter value: 30
Inserted: 30

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 3
Queue elements: 10 20 30

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 2
Deleted: 10

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 3
Queue elements: 20 30
```

3.Write a C program using stack data structure to simulate managing the history of web pages visited.

code:

```
#include <stdio.h>
#include <string.h>

#define MAX 5

char stack[MAX][50];
int top = -1;

void visit()
{
    char page[50];

    if (top == MAX - 1)
    {
        printf("History Full\n");
        return;
    }

    printf("Enter web page: ");
    scanf("%s", page);

    top++;
```

```
    strcpy(stack[top], page);

    printf("Visited: %s\n", page);
}

void back()
{
    if (top == -1)
    {
        printf("No history\n");
        return;
    }

    printf("Back from: %s\n", stack[top]);
    top--;

    if (top >= 0)
        printf("Current page: %s\n", stack[top]);
}

void display()
{
    int i;

    if (top == -1)
    {
        printf("History is empty\n");
        return;
    }

    printf("History:\n");

    for (i = top; i >= 0; i--)
        printf("%s\n", stack[i]);
}

int main()
{
    int choice;

    while (1)
    {
        printf("\n1. Visit\n2. Back\n3. Display\n4. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                visit();
                break;

            case 2:
                back();
```

```
                break;

            case 3:
                display();
                break;

            case 4:
                return 0;

            default:
                printf("Invalid choice\n");
        }
    }
}
```

output:

```
1. Visit
2. Back
3. Display
4. Exit
Enter choice: 1
Enter web page: google.com
Visited: google.com

1. Visit
2. Back
3. Display
4. Exit
Enter choice: 1
Enter web page: youtube.com
Visited: youtube.com

1. Visit
2. Back
3. Display
4. Exit
Enter choice: 1
Enter web page: github.com
Visited: github.com

1. Visit
2. Back
3. Display
4. Exit
Enter choice: 3
History:
github.com
youtube.com
google.com

1. Visit
2. Back
```

```
3. Display
4. Exit
Enter choice: 2
Back from: github.com
Current page: youtube.com
```

4. Write a C program to sort a stack using temporary stack

code:

```
#include <stdio.h>
#define MAX 5
int stack[MAX], temp[MAX];
int top = -1, ttop = -1;
void push(int x)
{
    stack[++top] = x;
}
int pop()
{
    return stack[top--];
}
void sortStack()
{
    int x;
    while (top != -1)
    {
        x = pop();
        while (ttop != -1 && temp[ttop] > x)
            stack[++top] = temp[ttop--];
        temp[++ttop] = x;
    }
    while (ttop != -1)
        stack[++top] = temp[ttop--];
}
int main()
{
    push(30);
    push(10);
    push(40);
    push(20);
    push(50);
    sortStack();
    printf("Sorted Stack: ");
    while (top != -1)
        printf("%d ", pop());
    return 0;
}
```

output:

```
Sorted Stack: 10 20 30 40 50
```

5.The Stock Span problem is a financial problem where we have a series of daily price quotes for a stock. The span of the stock price on a given day is defined as the maximum number of consecutive days leading up to that day (including that day) where the stock price is less than or equal to the price on that day. Take n to calculate the span of the stock price for all days. Implement using stack.

code:

```
#include <stdio.h>
int main()
{
    int price[] = {100, 80, 60, 70, 60, 75, 85};
    int stack[7], span[7];
    int top = -1;
    int n = 7;
    int i;
    for (i = 0; i < n; i++)
    {
        while (top != -1 && price[stack[top]] <= price[i])
            top--;
        if (top == -1)
            span[i] = i + 1;
        else
            span[i] = i - stack[top];
        stack[++top] = i;
    }
    printf("Span: ");
    for (i = 0; i < n; i++)
        printf("%d ", span[i]);
    return 0;
}
```

output:

```
Span: 1 1 1 2 1 4 6
```

6.Given an array of integers, for each element find the nearest element to its left that has a frequency greater than the current element. If no such element exists, get value -1. Implement using stack.
Input: [2, 1, 1, 3, 2, 1, 3]
output:

code:

```
#include <stdio.h>

int main()
{
    int a[] = {2, 1, 1, 3, 2, 1};
    int n = 6;
    int stack[6], top = -1;
    int freq[10] = {0};
    int ans[6];
```

```
    for (int i = 0; i < n; i++)
        freq[a[i]]++;

    for (int i = n - 1; i >= 0; i--)
    {
        while (top != -1 && freq[stack[top]] <= freq[a[i]])
            top--;

        if (top == -1)
            ans[i] = -1;
        else
            ans[i] = stack[top];

        stack[++top] = a[i];
    }

    printf("Output: ");
    for (int i = 0; i < n; i++)
        printf("%d ", ans[i]);

    return 0;
}
```

output:

```
 1 -1 -1 2 1 -1
```

7. Given a pattern containing only I's and D's. 'I' for increasing and 'D' for decreasing. Devise an algorithm using stack to print minimum number following that pattern. Digits from 1â€“9 and digits can't repeat.
I/P:D O/P:21
I/P:DIDI O/P:21435

code:

```
#include <stdio.h>
#include <string.h>
int main()
{
    char pattern[100];
    int stack[100], top = -1;
    int num = 1;
    scanf("%s", pattern);
    for (int i = 0; i <= strlen(pattern); i++)
    {
        stack[++top] = num++;
        if (pattern[i] == 'I' || pattern[i] == '\0')
        {
            while (top >= 0)
                printf("%d", stack[top--]);
        }
    }
    printf("\n");
```

```
    return 0;
}
```

output:

```
2
D
21
DIDI
21435
```

8. Given an array of digits (values are from 0 to 9), find the minimum possible sum by forming two numbers from digits of an array. All digits of given array must be used to form two numbers (for optimal result).use queue for implementation

I/P: [6, 8, 4, 5, 2, 3, 3]
O/P: 604 (i.e., 358 and 246)

code:

```
#include <stdio.h>
int main()
{
    int a[100], n;
    int i, j, temp;
    int num1 = 0, num2 = 0;
    printf("Enter number of digits: ");
    scanf("%d", &n);
    printf("Enter digits: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    for (i = 0; i < n - 1; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            if (a[i] > a[j])
            {
                temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }
    }
    for (i = 0; i < n; i++)
    {
        if (i % 2 == 0)
            num1 = num1 * 10 + a[i];
        else
            num2 = num2 * 10 + a[i];
    }
    printf("Minimum sum: %d\n", num1 + num2);
    return 0;
}
```

output:

```
Enter number of digits: 6
Enter digits: 6 8 4 5 2 3
Minimum sum: 604
```

9. Given an array and a positive integer k, find the first negative integer for each window (contiguous subarray) of size k. If a window does not contain a negative integer, then print 0 for that window. Implement using queue.

```
I/P: [-8, 2, 3, -6, 1], k = 2
O/P: [-8, 0, -6, -6]
first negative integer for each window of size z
[-8,2]=-8
[2,3]=0
[3,-6]=-6
[-6,1]=-6
```

code:

```
#include <stdio.h>
int main()
{
    int a[100], queue[100];
    int n, k, front = 0, rear = -1;
    int i;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    printf("Enter window size: ");
    scanf("%d", &k);
    for (i = 0; i < n; i++)
    {
        if (a[i] < 0)
            queue[++rear] = i;
        if (i >= k - 1)
        {
            while (front <= rear && queue[front] < i - k + 1)
                front++;
            if (front <= rear)
                printf("%d ", a[queue[front]]);
            else
                printf("0 ");
        }
    }
    return 0;
}
```

output:

```
Enter number of elements: 5
Enter elements: -8 2 3 -6 1
Enter window size: 2
-8 0 -6 -6
```

10.Given a matrix of dimension m Ã— n where each cell in matrix can have values 0, 1, 2 which has the following meaning:

0: empty cell.
1: cell having fresh orange.
2: cell having rotten orange.

The task is to find the minimum time required so that all oranges become rotten. A rotten orange at index (i,j) can rot other fresh oranges which are its neighbours (up, down, left and right). If impossible to rot every orange, return -1. Implement using Queue for above.

Example

I/P:

```
2 1 0 2 1
1 0 0 2 1
1 0 0 2 1
```

O/P:

2

code:

```
#include <stdio.h>
int main()
{
    int a[10][10] = {
        {2, 1, 0, 2, 1},
        {1, 0, 0, 2, 1},
        {1, 0, 0, 2, 1}
    };
    int queue[100][2];
    int front = 0, rear = -1;
    int rows = 3, cols = 5;
    int time = 0;
    int i, j;
    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < cols; j++)
        {
            if (a[i][j] == 2)
            {
                rear++;
                queue[rear][0] = i;
                queue[rear][1] = j;
            }
```

```
    }
}
while (front <= rear)
{
    int size = rear - front + 1;
    int changed = 0;
    for (i = 0; i < size; i++)
    {
        int x = queue[front][0];
        int y = queue[front][1];
        front++;
        if (x > 0 && a[x - 1][y] == 1)
        {
            a[x - 1][y] = 2;
            queue[++rear][0] = x - 1;
            queue[rear][1] = y;
            changed = 1;
        }
        if (x < rows - 1 && a[x + 1][y] == 1)
        {
            a[x + 1][y] = 2;
            queue[++rear][0] = x + 1;
            queue[rear][1] = y;
            changed = 1;
        }
        if (y > 0 && a[x][y - 1] == 1)
        {
            a[x][y - 1] = 2;
            queue[++rear][0] = x;
            queue[rear][1] = y - 1;
            changed = 1;
        }
        if (y < cols - 1 && a[x][y + 1] == 1)
        {
            a[x][y + 1] = 2;
            queue[++rear][0] = x;
            queue[rear][1] = y + 1;
            changed = 1;
        }
    }
    if (changed)
        time++;
}
for (i = 0; i < rows; i++)
{
    for (j = 0; j < cols; j++)
    {
        if (a[i][j] == 1)
        {
            printf("-1");
            return 0;
        }
    }
}
```

```
    printf("%d", time);
    return 0;
}
```

output:

2